

Kodewala Academy

Transforming code into career

Core Java & Advance Java

Kodewala

Kodewala Academy

Overview of Java

What is Java?

A high-level, platform-independent, object-oriented programming language developed by Sun Microsystems (now owned by Oracle).

History and Evolution of Java:

Java was developed in 1991 and released in 1995

Versions from Java 1.0 to the latest LTS (e.g., Java 21).

LTS (Long Term Support) releases, meaning they receive extended support from Oracle and other vendors.

Why Do we use Java?

Use cases: Web development, mobile apps (Android), enterprise applications (large-scale software), game development, etc.

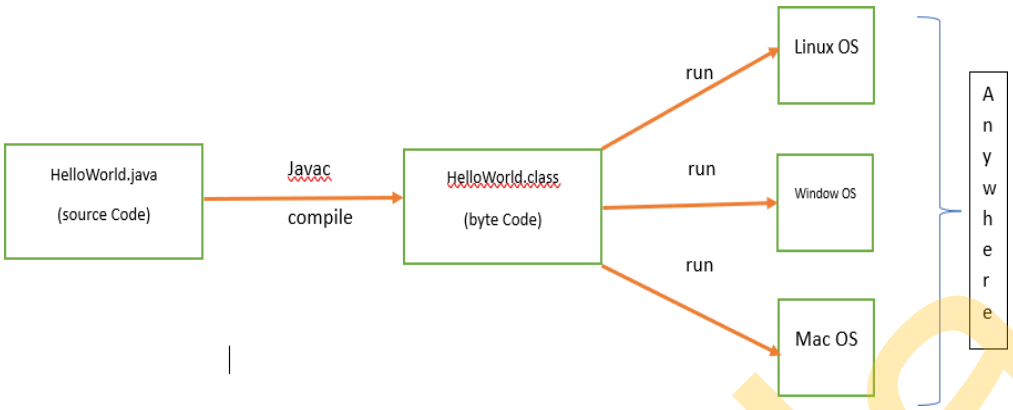
Note: Oracle Corporation acquired Sun Microsystems in 2010. Since then, Java has been licensed and maintained by Oracle Corporation.

Features of Java:

1. **Platform Independent:** Run code anywhere with JVM on any operating system (Window, Linux, Mac etc.).

   When we write and compile a Java program, the result is a .class file, which contains the **bytecode**. This bytecode is platform-independent, meaning it can be compiled on one operating system and executed on any other operating system. This concept is known as **WORA** (Write Once, Run Anywhere)

Kodewala



2. **Object-Oriented:** Java is an object-oriented programming (OOP) language, where everything is treated as an object. OOP means organizing software into different objects that combine both data and behaviour.

   OOP simplifies software development and maintenance by following a set of principles, which are:

   Object: An instance of a class, containing both data and methods.

   Class: A blueprint for creating objects, defining their properties and behaviours.

   Inheritance: A way for one class to inherit properties and behaviours from another.

   Polymorphism: The ability for different objects to respond to the same method in their own way.

   Abstraction: Hiding complex implementation details, showing only essential features.

   Encapsulation: Bundling data and methods together, restricting access to the internal state of objects.

3. **Robust and Secure:** Java offers built-in **garbage collection**, **strong memory management**, and **runtime error checking** to enhance program efficiency and reliability.

4. **High Performance:** With Just-In-Time (JIT) compilation.

Kodewala

Industry Experience (Real-World Example)

- ✓ "**Write Once, Run Anywhere**" (**WORA**) principle has been a game-changer. Applications built on Windows servers can seamlessly run on Linux-based servers without modification.

- ✓ Java's built-in **garbage collection** and runtime error checks helped ensure smooth operation in applications that handle millions of transactions daily (e.g., payment gateways or banking systems).

Interview Experience

- ✓ If you have compiled the java code in window machine, would you be able to run on Linux?
- ✓ What is WORA, and how does Java achieve it?
- ✓ Explain the significance of Java being platform-independent.

Kodewala

Kodewala Academy

Your First Java Code: Hello World

Prerequisite: Installation of Java, Notepad++

Steps to Implement Java Program

- Write the java program (Creating the java class)
- Compile the program (Using javac command)
- Run / Execute the program (using java command)

Write the java program (Creating the java class)

Note: We will be creating the java program using Notepad++.

```
// This is you first Java program.
// FileName : "HelloWorld.java".

public class HelloWorld

{

public static void main(String args[])
    {

        // Your program begins with a call to main(String args[]) method
        // Prints "Welcome to the Kodewala!" to the command window.

        System.out.println("Welcome to the Kodewala!");

    }

}
```

1. Class Definition

This line uses the keyword **class** to declare that a new class is being defined.

```
public class HelloWorld

 {
 // Statements, Methods, Logics etc.
 }
```

2. HelloWorld

"HelloWorld" is the name of the class and in java everything will be between the opening curly brace "**{**" and the closing curly brace "**}**".

3. main() Method

In the Java, every application must have a **main** method. The main method is the entry point of your Java application, and it is mandatory in a Java program.

```
public static void main(String[] args)
```

4. Explanation of the main method Syntax

- ✓ ***public:*** JVM can call the method from anywhere with in Java universe.
- ✓ ***static:*** The main method is to be called without an object creation. The modifiers are public and static can be written in either order.
- ✓ ***void:*** The main method does not return anything.
- ✓ ***main()****:* Name configured in the JVM. The main method must be inside the class definition. The JVM executes the codes starting always from the main function.

1. ***String[]*** : The main method accepts a single argument, an array of elements of type String. This would be used to receive a command line argument for the application.

5. Code inside the main() method

```
System.out.println("Welcome to the Kodewala!");
```

The statement can be broken into 3 parts which can be understood separately:

1. **System**: It is a predefined java class.
2. **out**: This is an instance of PrintStream type, which is a public and static member field of the System class.
3. **println():** This is a method in PrintStream. It prints any argument passed to it and adds a new line to the output.

Kodewala

Now Let's compile and run the program.

1. **Compiling the Program**

   - ✓ After successfully setting up the environment, we can open a terminal in Mac/Windows/Unix and go to the directory where the file, "HelloWorld.java" is present.
   - ✓ Now, to compile the "HelloWorld" program, execute the compiler – **javac**, to specify the name of the source file on the command line, as shown below.

     <You program directory> javac HelloWorld.java

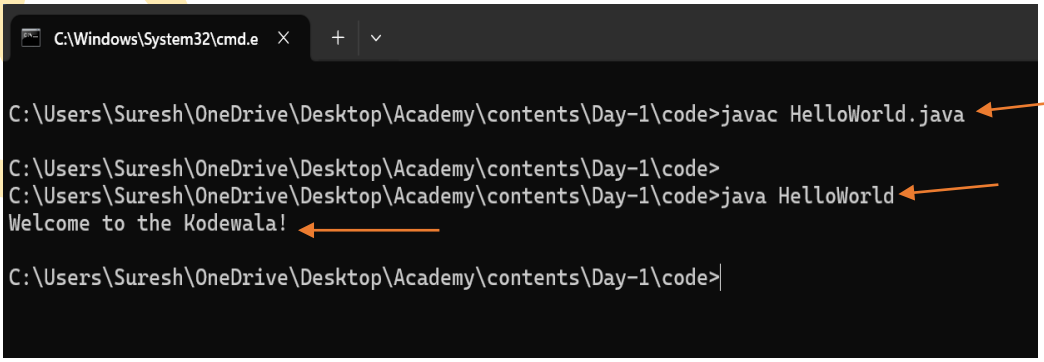
   - ✓ Once you run the above command, the compiler compile the code and create "HelloWorld.class" in the same directory. The .class contains the bytecode version of the program.

2. **Running / Executing the Program**

   - ✓ Now, to execute our program, **JVM** (Java Virtual Machine) needs to be called using Java, specifying the name of the **class** file on the command line, as shown below.

     <You program directory> java HelloWorld

   - ✓ This will print "Welcome to the Kodewala!" to the command screen.
   - ✓ Compile and run in window



   - ✓ With this you are able to code and run the Java HelloWorld program. Fantastic New!

Point To Note: "In java file name and class name should be the same". When the above law is violated a compiler error message will appear as below. We will discuss about the file name rules in next sessions. As of now just remember the rule: "**Class Name should be same as file Name**"

Kodewala

Kodewala Academy

Industry Experience

- ✓ Debugging the code and understanding the error and exception.
- ✓ Improve debugging skills.
- ✓ Run without main method.
- ✓ Verify the generate bytecode / class files.

Interview Experience

- ✓ Why main method is static?
- ✓ Can we run the program without main method?
- ✓ What is the command used to compile and run the java program?
- ✓ What is the purpose the String args[] ?
- ✓ Explain the System.out.println()
- ✓ File Name vs Class name
- ✓ How many class files will be created for each java file?

Kodewala

Kodewala Academy

JDK, JRE, and JVM

JDK

Definition: JDK stands for Java Development Kit, which is a complete toolkit for Java development. It includes everything needed to develop Java applications.

The main components of the JDK (Java Development Kit) are:

- ✓ **JRE**: JDK includes the Java Runtime Environment, which provides the runtime libraries and JVM required to run Java programs.
- ✓ **Compilers**: JDK contains a compiler (javac) that translates Java source code into bytecode.
- ✓ **Development Tools**: JDK provides additional tools such as javap (disassembler), javadoc (document generator), and jdb (debugger), which are essential for building, debugging, and documenting Java programs.
- ✓ **Libraries**: JDK comes with a set of standard libraries (Java API) that provide functionality like I/O operations, networking, database connections, etc.
- ✓ **JVM**: JDK includes the Java Virtual Machine, which is responsible for running the bytecode.

Purpose: JDK is primarily used by developers to write, compile, and debug Java applications. It is required for anyone developing Java programs from scratch.

JRE (Java Runtime Environment):

Definition: JRE stands for Java Runtime Environment. It provides the necessary environment for running Java applications but does not include development tools.

The main components of the JRE (Java Runtime Environment) are:

- ✓ **JVM**: JRE includes the Java Virtual Machine, which interprets and runs the compiled Java bytecode.
- ✓ **Libraries**: JRE provides the core set of libraries required to run Java programs, such as the Java standard library (java.lang, java.util, etc.).
- ✓ **Native Libraries**: JRE also includes platform-specific native libraries needed for running Java applications.

Purpose: JRE is used by users or systems that only need to run Java applications, not develop them. It does not include compilers, debuggers, or other development tools that are part of the JDK.

Kodewala

Kodewala Academy

JVM

- ✓ JVM stands for Java Virtual Machine, which is not visible to the user.
- ✓ It is a part of the JRE (Java Runtime Environment).
- ✓ JVM is responsible for executing Java programs.
- ✓ It allocates memory required for program execution.
- ✓ JVM de-allocates memory when it's no longer needed.
- ✓ It converts bytecode into a machine-understandable format.
- ✓ JVM will be created when user execute the “java” command and it will be destroyed once program is executed.

Summary of Differences:

- ✓ **JDK** is a **development toolkit** that includes tools for compiling and debugging Java applications, and it contains **JRE**.
- ✓ **JRE** is the **runtime environment** for running Java applications; it includes the **JVM** but does not have development tools like compilers.
- ✓ **JVM** is the **virtual machine** that executes Java bytecode and manages memory during execution, and it is a key component of **JRE**.

Industry Experience

- ✓ The JRE is sufficient whenever the need is limited to running Java applications without development or debugging requirements.
- ✓ JDK is required for development

Interview Experience

- ✓ Can we install JRE without JDK?
- ✓ An application runs fine on one system but fails on another. Both have the JRE installed. What could be the problem?

Kodewala

Command Line Arguments in Java

About it

Java command-line argument is an argument, passed at the time of running the Java program (java HelloWorld “arg1” “arg2”).

In Java, the command line arguments passed from the console can be received in the Java program and they can be used as input to perform various activities. The users can pass the arguments during the execution bypassing the command-line arguments inside the main() method. This is used to make the program flexible and dynamic.

How to pass command line Args?

We need to pass the arguments as space-separated values. We can pass both strings and primitive data types (int, double, float, char, etc) as command-line arguments. These arguments convert into a string array and are provided to the main (String args[]) function as a string array argument. User need to convert back to desired format as per need.

When user supply the command line args, JVM will wraps up these command-line arguments into the args[ ] array that we pass into the main() method.

Write a program to read the command line arguments

```
public class CmdArgs
{

public static void main(String args[])
{
    String name = args[0];
    String location = args[1];
    System.out.println("Welcome to the "+name);
    System.out.println(name + " is located in "+location);
}

}
```

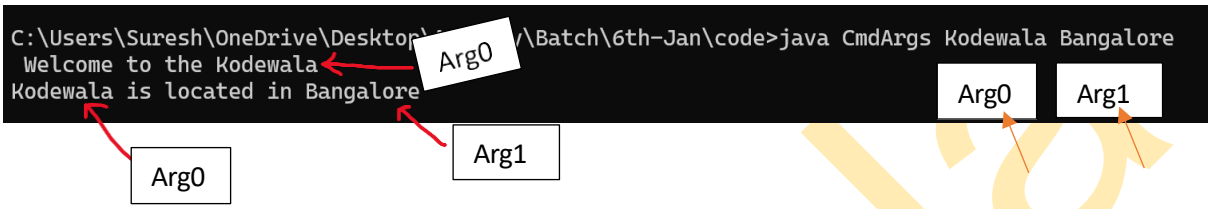
kodewala

Kodewala Academy

How to pass command line arguments?

While running the java program, user need to pass the arguments as below.

<location>java <class name> <arg0> <arg1> <arg2> <arg…n>



Important Points to Remember:

- ✓ **String Input:** It takes input in the space separated format.
- ✓ **Dynamic-input:** Allows user to pass dynamic inputs.
- ✓ **Length**: We can check the length using .length property of the array.
- ✓ **Indexes**: We can take the values just using the indexes args[0] args[1] args[2] args[n]

Industry Experience

- ✓ If you are developing any application like reading file, user can pass file name as a command line argument.
- ✓ If you are developing the simple application like calculator, user can pass input using command line arguments.
- ✓ If you feel that any input may change in future, like data, filename etc. Use the command line arguments.

Interview Experience

- ✓ If you are developing the utility to read the file. What would be your approach to read the path, considering the path may change in future
- ✓ What will happed if you are readding the command line arguments but not passing them?
- ✓ How will you make sure that command line arguments are passed and in case user missed to pass the args, throw the warning. i.e “Please enter the input”

Kodewala

Kodewala Academy

Java Identifiers

About it

An identifier in Java is the name given to **Variables, Classes, Methods, Packages, Interfaces,** etc. These are the unique names and every Java Variables must be identified with unique names.

Let's understand with simple example

```
public class Kodewala
{
   public static void main(String[] args)
   {
      int balance = 1200;
      String location ="Bangalore";
   }
}
```

In the above program, we have **total 6** identifiers as below.

- ✓ **Kodewala**: Class name
- ✓ **main**: Method name
- ✓ **String**: Predefined Class name
- ✓ **args**: Variable name
- ✓ **balance**: Variable name
- ✓ **location**: Variable name

In Java, there are rules for the identifiers

Java identifiers must adhere to the following rules, otherwise you will get compile time error/failure.

1. **Allowed Characters:** Identifiers can only include alphanumeric characters ([A-Z], [a-z], [0-9]), the dollar sign ($), and the underscore (_). For example, "Kodewala#" is not valid because it contains the special character #.
2. **Starting Character:** Identifiers cannot begin with a digit ([0-9]). For instance, "12Kodewala" is invalid because it starts with a number.
3. **Case Sensitivity:** Java identifiers are case-sensitive, meaning Example and example would be considered different.

Kodewala

4. **Length:** There is no restriction on the maximum length of an identifier. However, it is recommended to keep the length between 4 and 15 characters for better readability and maintainability.
5. **Reserved Words:** Reserved words in Java cannot be used as identifiers. For example, the statement int **for** = 1230; is invalid because while is a reserved word. Java has 50+ reserved words that cannot be used as identifiers.

Here are some examples of identifiers in Java.

Valid	Invalid	
FirstName	**Test Name**	// contains a space
FIRSTNAME	**34kodewala**	// Begins with a digit
firstname	**ca+t**	// plus sign is not an alphanumeric character
y	**variable-5**	// hyphen is not an alphanumeric character
j	**last_&_first**	// ampersand is not an alphanumeric character
y1		
firstname		
$firstname		
total_sum		
code123		

Kodewala

Reserved Words in Java

In programming languages, certain words are reserved to represent specific functionalities defined by the language. These are known as reserved words.

Category	Keywords
Primitive Data Types	`boolean`, `byte`, `char`, `double`, `float`, `int`, `long`, `short`
Control Flow Statements	`if`, `else`, `switch`, `case`, `default`, `while`, `do`, `for`, `break`, `continue`, `return`
	`try`, `catch`, `finally`, `throw`, `throws`
Modifiers	`abstract`, `final`, `private`, `protected`, `public`, `static`, `synchronized`, `transient`
	`volatile`, `strictfp`
Class and Object	`class`, `interface`, `enum`, `extends`, `implements`, `this`, `super`, `instanceof`, `new`
Package Related	`import`, `package`
Exception Handling	`assert`
Multi-threading	`synchronized`, `volatile`
Special Purpose	`const` (reserved but not used), `goto` (reserved but not used), `null`, `true`, `false`
Miscellaneous	`native`

Important Points to Remember:

Always write code that is easy to read and self-explanatory. Let's understand with below example.

```java
public class FundTransfer {

    // Method to transfer funds between accounts
    public void transfer(String fromAccount, String toAccount, double amount) {
        if (amount > 0) {
            System.out.println("Transferred INR " + amount + " from " + fromAccount + " to " + toAccount);
        } else {
            System.out.println("Invalid transfer amount.");
        }
    }

    public static void main(String[] args) {
        FundTransfer fundTransfer = new FundTransfer();

        // Transfer funds from account 1234567 to account 4567893
        fundTransfer.transfer("1234567", "4567893", 500.00);
    }
}
```

Class Name: *FundTransfer*

- ✓ The class name is concise and descriptive. It clearly indicates that the class is designed to handle the process of transferring funds between accounts.
- ✓ **Why it matters:** The name provides clarity and helps other developers understand the class's purpose immediately, improving the readability of the code.

Method Name: *transfer*

- ✓ The method name transfer directly describes the action performed by the method - transferring money from one account to another.
- ✓ **Why it matters**: The method name is intuitive and easy to understand, making the code more readable and maintainable. It follows the principle of "self-describing" method names, where the method name communicates what it does.

Parameters: *fromAccount, toAccount, amount*

- ✓ fromAccount and toAccount: These parameters represent the account numbers involved in the transfer. Their names are straightforward and clearly indicate their purpose.
- ✓ amount: This parameter represents the amount to be transferred. It is also named clearly to avoid ambiguity.
- ✓ **Why it matters:** Descriptive parameter names enhance the readability of the code and help other developers understand what values are expected by the method.

Kodew

Industry Experience

- ✓ Always give meaningful name for variable and classes, methods.
- ✓ If you are providing the meaningful name, it will be more readable and easier to debug the issues.

Interview Experience

- ✓ There would be question about writing the clean code and maintainability.

Class work

- ✓ Write a program to display employee details provided through command-line arguments, ensuring adherence to Java naming conventions.

Assignment

- ✓ Write a program demonstrating the use of various conventions and rules for naming identifiers.

s

Kodewala